Conversation Chat OCR & Classification

Problem Brief, Tech Stack, and End-to-End Pipeline

How to extract conversation text from image-based inbox PDFs/PNGs, structure it into messages, and classify intent with a lean local pipeline.

Stack: Rust · Tesseract · TensorFlow

Document purpose: design overview (no implementation code)

1. The Problem

The workspace contains Fiverr Inbox exports: large PDFs (some over 100 MB) and at least one PNG screenshot. These files mostly capture **conversation UI as images** inside PDF pages—not selectable digital text. Standard PDF text extraction (copy/paste, pdftotext, naive parsers) fails or returns empty/garbage output.

What we need

- **Extract** readable text from image-based PDFs and PNGs.
- **Structure** that text into chat messages (speaker, time, body) instead of a raw OCR soup.
- **Classify** messages by intent (e.g. revision, delivery, price, dispute), then roll up labels for each conversation/file.

Why this is hard

- Pages are screenshots: OCR must recover characters from pixels.
- Chat UI adds noise: Online status, buttons, badges, sidebars, system notices.
- Speaker and timestamps are layout cues, not plain paragraphs.
- Long exports make naive high-DPI OCR of every page expensive in CPU, RAM, and time.

2. Tech Stack

The chosen stack splits work by strength: Rust for fast local extraction, Tesseract for character recognition, TensorFlow for text intent classification.

Layer	Tool	Responsibility
Orchestration & I/O	Rust	Render PDF/PNG pages, preprocess images, call OCR, group lines into messages, cache JSON, batch files.
Character recognition	Tesseract	Turn page images into characters, words, and lines (with bounding boxes when needed).
Classification	TensorFlow	Train/serve a small text model that maps message text → intent label + confidence.
Inference bridge	Python TF or ONNX	Train in Python/TF; serve via a small Python process or export ONNX and run from Rust (ort).

Table 1. Stack roles across the pipeline.

What we deliberately avoid by default

- Vision LLMs on every page (accurate for layout, but costly and privacy-sensitive).
- Running TensorFlow natively inside Rust for training (weak ergonomics); prefer Python train → ONNX/Python serve.
- Re-OCRing the same file on every experiment—always cache structured JSON.

3. Entire Pipeline

Data flows from pixels to labels. Tesseract only recognizes characters. Rust turns OCR output into conversation structure. TensorFlow classifies cleaned message text.

PDF/PNG → page images → preprocess → **Tesseract OCR** → raw lines → **Rust message structuring** → cached JSON → **TensorFlow intents** → conversation rollup

Figure 1. High-level data path from file to labels.

Stage-by-stage detail

Stage 1 — File to page images (Rust)

Open each PDF or PNG. Render PDF pages at 150–200 DPI (or load the PNG). Process page-by-page so large 100 MB exports do not stay fully decoded in memory.

Stage 2 — Preprocess (Rust)

Optional grayscale, contrast, and crop to the chat pane. Improves OCR accuracy and reduces work on sidebars and chrome.

Stage 3 — Character recognition (Tesseract)

OCR emits characters → words → lines, ideally with bounding boxes. Output is still messy UI text (status labels, timestamps, mixed speakers).

Stage 4 — Lines to structured messages (Rust)

Sort by reading order. Drop chrome via rules. Group lines into messages using gaps, timestamps, name patterns, and left/right layout cues. Emit speaker, time, and text. Cache JSON per file.

Stage 5 — Classification inputs

Each message text (optionally with a short neighbor context) becomes one classifier example. Obvious Fiverr system notices can be rule-labeled and skip the model.

Stage 6 — Intent classification (TensorFlow)

A small text model predicts intent + confidence, e.g. greeting, scope, price, revision, delivery, dispute, system, other.

Stage 7 — Conversation rollup

Aggregate message labels per file/thread: intent histogram, risk flag if dispute/refund language appears, and fields useful for search or review. Final artifact is JSON + labels—not page bitmaps.

4. Example Data Shapes

After OCR (raw lines — illustrative)

```
Online
BuyerName
Hi, can you revise the logo colors?
11:42 AM
You
Sure, I will update and deliver today.
11:45 AM
Delivery
Your order was delivered
```

After structuring (messages JSON — illustrative)

```
{ "speaker": "buyer", "time": "11:42 AM", "text": "Hi, can you revise the logo colors?" }
{ "speaker": "seller", "time": "11:45 AM", "text": "Sure, I will update and deliver
today." }
{ "speaker": "system", "time": null, "text": "Your order was delivered" }
```

After classification (labels — illustrative)

```
{ "text": "Hi, can you revise the logo colors?", "intent": "revision", "confidence": 0.91
}
{ "text": "Sure, I will update and deliver today.", "intent": "delivery", "confidence":
0.84 }
{ "text": "Your order was delivered", "intent": "system", "confidence": 1.0 }
```

5. Resource Profile

The architecture is not inherently too heavy if OCR is cached and DPI stays moderate. Cost is dominated by rasterization and Tesseract—not by classifying short message strings.

- **First run** on a large PDF: minutes-scale (OCR dominates).
- **Later runs**: fast when structured JSON cache hits.
- **TF inference**: cheap relative to OCR; CPU is enough for a small text model.
- **GPU**: optional for training only; not required for daily extraction/inference.
- **Guards**: 150–200 DPI, page batches, delete temp images after OCR, skip blank/duplicate pages.

6. Bottom Line

Treat Fiverr inbox PDFs as images. Use **Rust + Tesseract** to turn pixels into structured chat messages, cache that output, then use a **small TensorFlow text classifier** for message intents and a conversation-level rollup. Keep vision LLMs and high-DPI full-file OCR out of the default path so the system stays practical on a normal machine.

Source context: local Chat folder Fiverr Inbox PDF/PNG exports. This document describes the agreed design approach for discussion and planning.